

Estudio de Caso sobre Métodos de Aprendizaje Supervisado:

Análisis, Modelado y Evaluación de Técnicas de Clasificación

David Núñez Franco

Angel Leiva Abarca

Billy Cordero Porras

Abril 2026

Resumen

Los métodos supervisados constituyen una base fundamental del aprendizaje automático, al permitir construir modelos predictivos a partir de datos previamente etiquetados. En este estudio de caso se analizan y comparan distintos algoritmos de clasificación aplicados al conjunto de datos BankChurners, utilizando como variable objetivo el abandono de clientes. Se abordan los fundamentos teóricos y experimentales de K vecinos más cercanos (KNN), Árboles de Decisión, Random Forest, AdaBoost y XGBoost, considerando configuraciones estándar y variaciones de hiperparámetros. La evaluación se realiza mediante métricas como precisión global, F1-score, ROC AUC y matriz de confusión, con especial atención al desbalance de clases. Los resultados permiten identificar el modelo con mejor desempeño y analizar las variables más influyentes en la predicción del abandono.

Palabras clave: aprendizaje supervisado, clasificación, KNN, árboles de decisión, Random Forest, AdaBoost, XGBoost.

1. Introducción

El aprendizaje supervisado permite construir modelos predictivos a partir de datos etiquetados, donde cada observación posee variables predictoras y una variable objetivo conocida.

En problemas de clasificación, dicha variable objetivo es categórica, por lo que el modelo busca aprender patrones históricos para asignar una clase a nuevas observaciones. Este enfoque resulta especialmente útil en contextos donde se requiere anticipar comportamientos, identificar patrones y apoyar la toma de decisiones a partir de datos previamente observados.

En este estudio de caso se analizan distintos algoritmos supervisados de clasificación aplicados al conjunto de datos *BankChurners*, utilizando como variable objetivo el abandono de clientes. El laboratorio contempla modelos como K vecinos más cercanos (KNN), Árboles de Decisión, Random Forest, AdaBoost y XGBoost, los cuales representan enfoques diferentes dentro del aprendizaje supervisado. KNN se basa en la similitud entre instancias y depende de aspectos como el número de vecinos, la métrica de distancia y la regla de clasificación [5, 3]. Los Árboles de Decisión permiten construir reglas jerárquicas interpretables mediante nodos, ramas y hojas [2, 1]. Por su parte, Random Forest y los métodos de boosting combinan múltiples modelos base para mejorar la capacidad predictiva y reducir limitaciones propias de clasificadores individuales [8, 9].

A partir de estos algoritmos, el estudio compara configuraciones estándar y variaciones de hiperparámetros, evaluando su desempeño mediante métricas como precisión global, F1-score, ROC AUC y matriz de confusión. Además, se considera el desbalance de clases presente en el conjunto de datos, con el fin de interpretar los resultados más allá de la exactitud general y seleccionar el modelo con mejor comportamiento predictivo.

2. K vecinos más cercanos

El algoritmo K vecinos más cercanos, o KNN por sus siglas en inglés *K-Nearest Neighbors*, es un método clásico de clasificación supervisada basado en la cercanía entre observaciones. Su principio central consiste en asignar a una nueva instancia la clase predominante entre sus k vecinos más cercanos dentro del conjunto de entrenamiento. Para ello, calcula la distancia entre la observación de prueba y las observaciones etiquetadas, selecciona las k más próximas y aplica normalmente una regla de mayoría.

KNN se considera un método no paramétrico porque no supone una forma específica para la distribución de los datos. Además, es un algoritmo de aprendizaje perezoso o *lazy learning*, ya que no construye un modelo explícito durante el entrenamiento, sino que conserva los datos y realiza la clasificación cuando recibe una nueva instancia [4]. Esta característica facilita su implementación, pero puede aumentar el costo computacional cuando el conjunto de entrenamiento es grande.

El rendimiento de KNN depende principalmente de tres elementos: el valor de k , la métrica de distancia y la regla de clasificación. Un valor pequeño de k puede hacer que el

modelo sea sensible al ruido, mientras que un valor muy grande puede suavizar demasiado las fronteras de decisión. La métrica de distancia, como la euclidiana, Manhattan, Minkowski o Mahalanobis, define cómo se mide la similitud entre observaciones. Por ello, cuando las variables numéricas poseen escalas diferentes, la estandarización resulta necesaria para evitar que ciertas variables dominen el cálculo de distancia.

Uno de los principales retos del algoritmo es seleccionar adecuadamente el valor de k . Zhang et al. [5] señalan que usar un mismo valor para todas las muestras puede ser limitado, ya que distintas regiones del espacio de datos pueden requerir diferentes cantidades de vecinos. Además, la búsqueda de vecinos puede convertirse en un cuello de botella, pues el algoritmo compara cada nueva instancia contra el conjunto de entrenamiento [4]. Aun así, KNN sigue siendo ampliamente utilizado por su simplicidad y buen desempeño. Para el laboratorio, se justifica evaluar distintos valores de k , aplicar estandarización y comparar resultados mediante matriz de confusión, precisión global, error global y precisión por categoría.

3. Análisis experimental del algoritmo KNN

En esta sección se presenta el análisis experimental del algoritmo KNN aplicado al conjunto de datos *BankChurners*, el cual contiene información de 810 clientes bancarios con 14 variables predictoras y una variable objetivo binaria (*Attrition_Flag*) que indica si un cliente abandonó el servicio (*Attrited Customer*) o permanece activo (*Existing Customer*). El análisis se desarrolló utilizando la clase `Supervisado` del paquete pythónico construido durante el curso.

3.1. Preparación de los datos

El conjunto de datos original contiene variables numéricas y categóricas. La preparación incluyó los siguientes pasos: eliminación de la columna de identificación (ID), codificación de la variable objetivo mediante `LabelEncoder` (0: *Attrited Customer*, 1: *Existing Customer*), y codificación numérica de las variables categóricas (`Gender`, `Education_Level`, `Marital_Status`, `Income_Category`).

Dado que KNN es un algoritmo basado en distancias, resulta imprescindible estandarizar las variables predictoras para evitar que aquellas con mayor escala dominen el cálculo de proximidad [3]. Se aplicó `StandardScaler` para normalizar todas las variables a media cero y desviación estándar unitaria. La partición de los datos se realizó con una proporción de 75 % para entrenamiento y 25 % para prueba, con estratificación sobre la variable objetivo

para mantener la proporción de clases en ambos subconjuntos.

Es importante señalar que el conjunto de datos presenta un desbalance de clases significativo: 687 clientes activos (84.8 %) frente a 123 clientes que abandonaron (15.2 %). Esta característica incide directamente en las métricas de evaluación, particularmente en la precisión por categoría de la clase minoritaria.

3.2. Configuración estándar de KNN

La configuración estándar de KNN se definió con los valores por defecto del clasificador `KNeighborsClassifier` de scikit-learn: $k = 5$ vecinos, algoritmo de búsqueda `auto`, pesos uniformes (`uniform`) y distancia euclidiana (`minkowski` con $p = 2$). Esta configuración constituye el modelo base contra el cual se comparan las variaciones posteriores.

Los resultados del modelo estándar se muestran en la Tabla 1.

Cuadro 1: Resultados del modelo KNN estándar ($k = 5$, uniforme, euclidiana).

Métrica	Valor
Precisión Global	0.8670
Error Global	0.1330
Precisión – Attrited Customer (clase 0)	0.3226
Precisión – Existing Customer (clase 1)	0.9651

La precisión global de 86.70 % puede parecer aceptable, pero la precisión de la clase minoritaria (*Attrited Customer*) es de apenas 32.26 %, lo que indica que el modelo estándar tiene dificultades para identificar correctamente a los clientes que abandonan. Esto se explica por el desbalance de clases y la regla de mayoría con pesos uniformes.

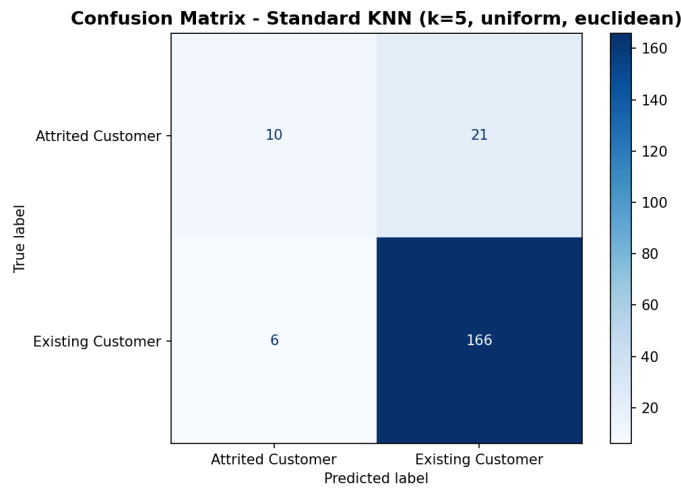


Figura 1: Matriz de confusión del modelo KNN estándar ($k = 5$, uniforme, euclidiana).

3.3. Variaciones de configuración

Se evaluaron sistemáticamente 480 configuraciones distintas del algoritmo KNN, combinando los siguientes hiperparámetros:

- **Número de vecinos (k):** de 1 a 20.
- **Algoritmo de búsqueda:** auto, ball_tree, kd_tree y brute.
- **Esquema de pesos:** uniform (voto igualitario) y distance (ponderación inversa a la distancia).
- **Métrica de distancia:** euclidiana ($p = 2$), Manhattan ($p = 1$) y Minkowski ($p = 3$).

La Figura 2 muestra la evolución de la precisión global conforme varía k para cada combinación de métrica y esquema de pesos.

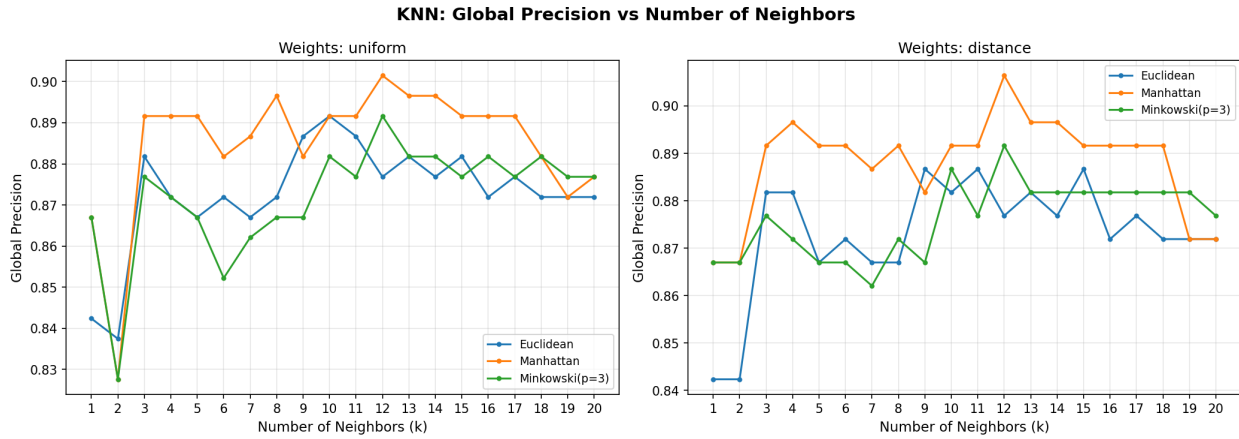


Figura 2: Precisión global en función del número de vecinos (k) para distintas métricas de distancia y esquemas de pesos.

Se observa que el esquema de pesos **distance** tiende a producir mejores resultados que **uniform**, especialmente para valores de k intermedios. Esto concuerda con la literatura, ya que la ponderación por distancia otorga mayor influencia a los vecinos más cercanos, lo cual resulta beneficioso cuando las fronteras de decisión no son uniformes [3].

La Figura 3 presenta un mapa de calor con la precisión global para cada combinación de k y métrica con pesos por distancia.

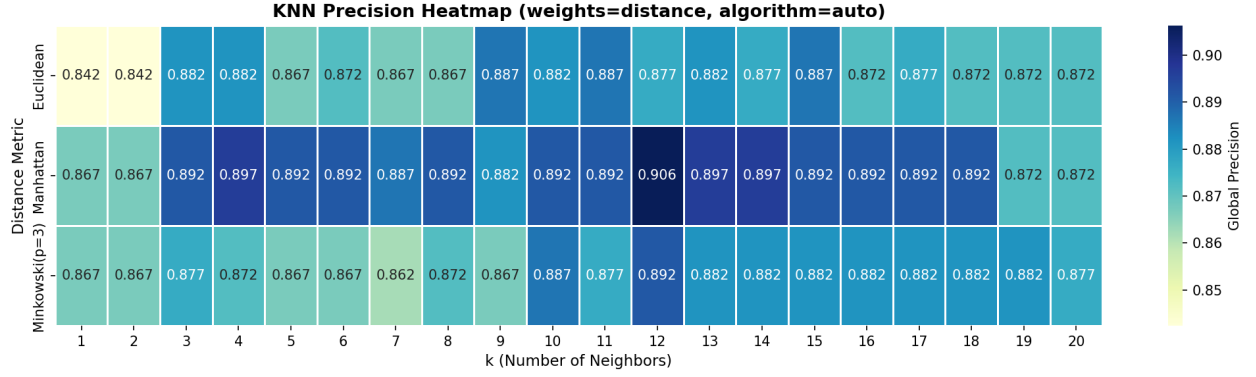


Figura 3: Mapa de calor de precisión global por k y métrica de distancia (weights=distance, algorithm=auto).

El mapa de calor revela que la distancia Manhattan tiende a producir valores ligeramente superiores para varios valores de k , con un pico notable en $k = 12$.

Respecto al algoritmo de búsqueda, la Figura 4 muestra que para un mismo valor de k y métrica, los cuatro algoritmos producen resultados idénticos, lo cual es esperable ya que solo difieren en la estrategia de búsqueda, no en el resultado de la clasificación.

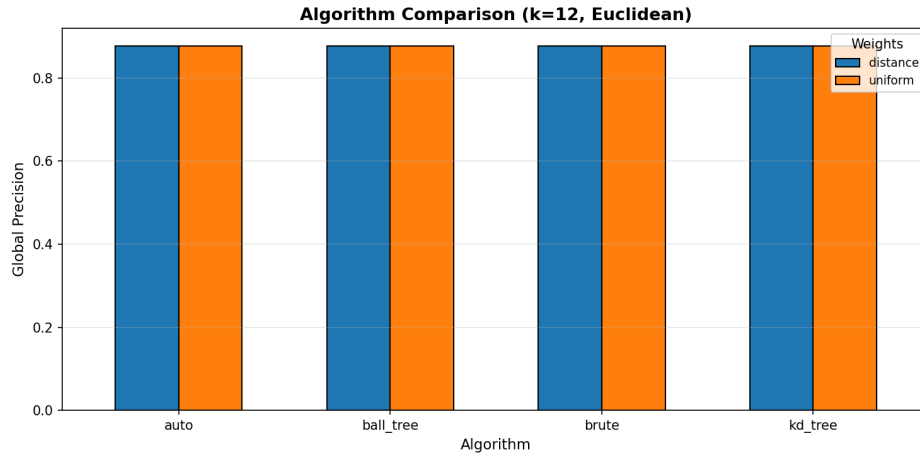


Figura 4: Comparación de algoritmos de búsqueda para el mejor k con distancia euclidiana.

3.4. Resumen del rendimiento por k

La Figura 5 sintetiza el rendimiento de KNN agrupado por valor de k , mostrando la mejor, peor y media de precisión global junto con la desviación estándar.

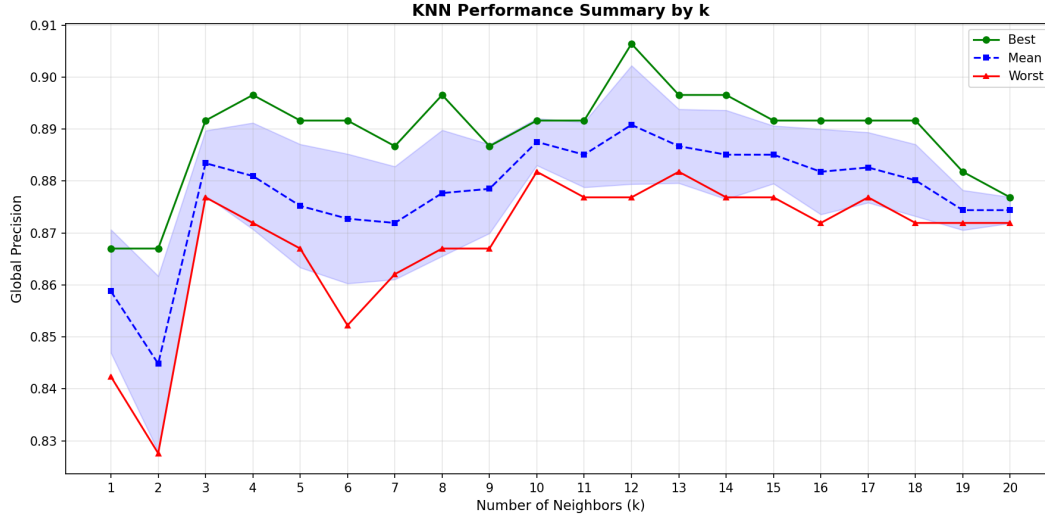


Figura 5: Resumen del rendimiento de KNN por valor de k : mejor, media y peor precisión global.

Se observa que valores de k muy pequeños (1–2) producen mayor variabilidad y menor rendimiento promedio, mientras que el rango $k \in [10, 14]$ presenta los mejores resultados. Para $k > 15$, la precisión comienza a descender ligeramente, lo cual concuerda con la idea de que un k excesivo suaviza las fronteras de decisión y diluye la señal de la clase minoritaria.

3.5. Selección del mejor modelo

La Figura 6 presenta las diez mejores configuraciones ordenadas por precisión global.

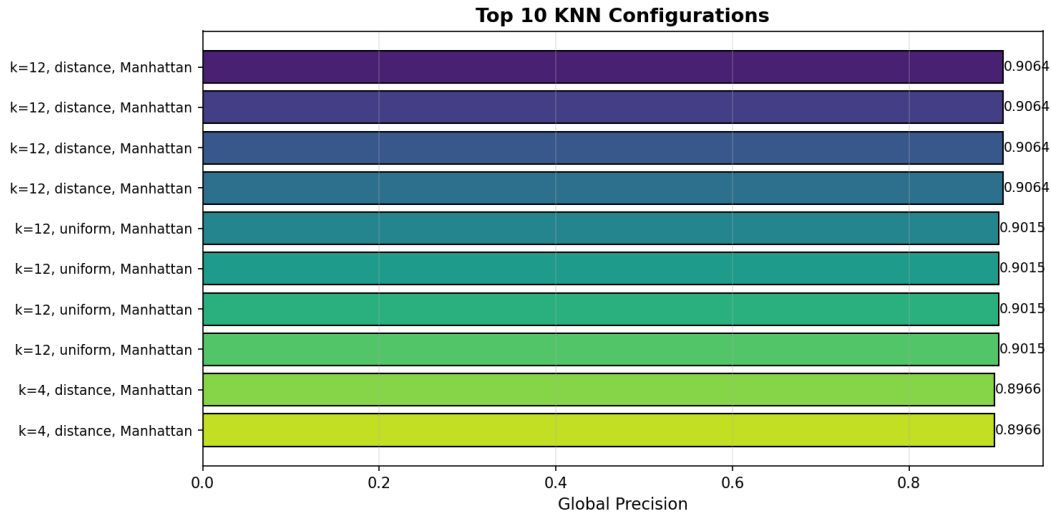


Figura 6: Top 10 configuraciones de KNN por precisión global.

El mejor modelo corresponde a la configuración $k = 12$, pesos por distancia (**distance**),

métrica Manhattan ($p = 1$) y algoritmo `auto`, con una precisión global de **90.64 %**. Dado que los cuatro algoritmos de búsqueda producen el mismo resultado, se selecciona `auto` por eficiencia computacional.

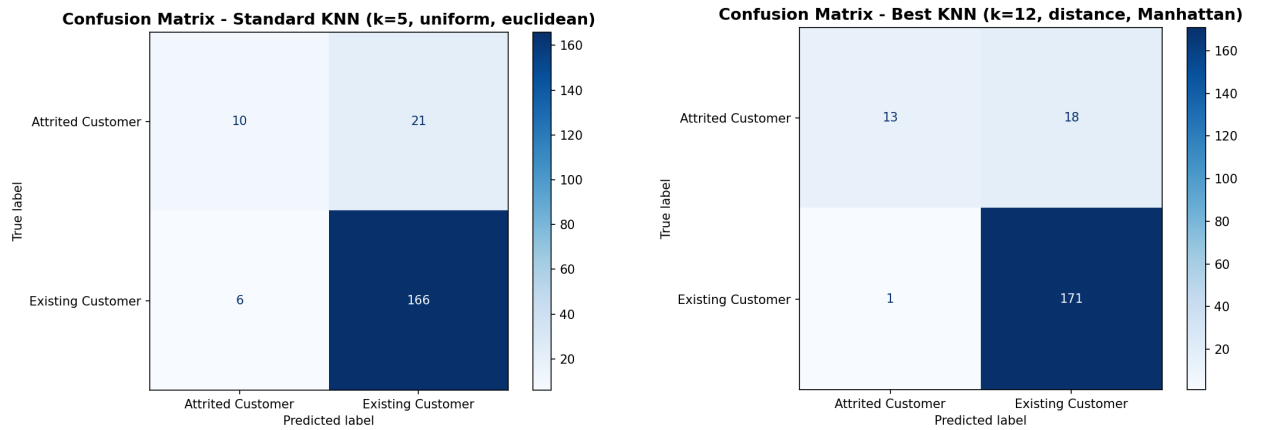
3.6. Análisis del mejor modelo

La Tabla 2 presenta la comparación entre la configuración estándar y la mejor configuración identificada.

Cuadro 2: Comparación entre la configuración estándar y la mejor configuración de KNN.

Métrica	Estándar ($k=5$, uniform, euclidiana)	Mejor modelo ($k=12$, distance, Manhattan)
Precisión Global	0.8670	0.9064
Error Global	0.1330	0.0936
Precisión – Attrited (clase 0)	0.3226	0.4194
Precisión – Existing (clase 1)	0.9651	0.9942

El mejor modelo logra una mejora de 3.94 puntos porcentuales en precisión global, reduciendo el error global de 13.30 % a 9.36 %. En particular, la precisión de la clase minoritaria (*Attrited Customer*) mejoró de 32.26 % a 41.94 %, y la de la clase mayoritaria (*Existing Customer*) alcanzó 99.42 %.



(a) Modelo estándar ($k=5$).

(b) Mejor modelo ($k=12$).

Figura 7: Matrices de confusión: configuración estándar vs. mejor modelo.

El reporte de clasificación detallado del mejor modelo indica una precisión ponderada de 91 %, con un *F1-score* de 0.95 para la clase mayoritaria y 0.58 para la clase minoritaria.

Si bien la detección de la clase *Attrited Customer* sigue siendo limitada (recall de 42 %), la mejora respecto al modelo estándar es significativa.

3.7. Interpretación de resultados

Los resultados obtenidos permiten formular las siguientes observaciones:

1. **Impacto de los pesos:** El uso de pesos por distancia (**distance**) mejora consistentemente la precisión respecto a pesos uniformes. Esto se debe a que los vecinos más cercanos reciben mayor influencia en la clasificación, lo cual resulta particularmente útil en datos con fronteras de decisión complejas [3].
2. **Efecto de la métrica:** La distancia Manhattan (L_1) produjo el mejor resultado. En espacios de alta dimensionalidad, la norma L_1 puede ser más robusta que la euclidiana (L_2), ya que esta última tiende a diluir las diferencias entre dimensiones [6].
3. **Selección de k :** El valor óptimo $k = 12$ se ubica en un rango intermedio que equilibra la sensibilidad al ruido de valores pequeños con la sobreesuavización de valores grandes. Zhang et al. [5] señalan que un k fijo para todo el espacio de datos es subóptimo y que idealmente cada muestra debería tener su propio k óptimo, lo cual motiva enfoques como CM-kNN.
4. **Desbalance de clases:** La baja precisión de la clase minoritaria (41.94 %) es un resultado esperado en problemas desbalanceados. Técnicas complementarias como sobremuestreo (SMOTE), submuestreo o ajuste del umbral de decisión podrían mejorar la detección de clientes que abandonan.
5. **Algoritmo de búsqueda:** Los cuatro algoritmos evaluados producen idénticos resultados de clasificación, ya que difieren únicamente en la eficiencia de la búsqueda de vecinos, no en la selección de los mismos. En la práctica, **auto** delega la elección al motor de scikit-learn según las características del conjunto de datos.

4. Árboles de Decisión

Los Árboles de Decisión son algoritmos supervisados que representan la clasificación mediante una estructura jerárquica de nodos, ramas y hojas. Cada nodo interno contiene una condición sobre una variable predictora, cada rama representa el resultado de dicha condición y cada hoja corresponde a una clase o decisión final. Su objetivo es dividir progresivamente el conjunto de datos en subconjuntos más homogéneos respecto a la variable objetivo.

El aprendizaje de un árbol se desarrolla como un proceso voraz, descendente y recursivo: en cada paso se selecciona el atributo que mejor separa las clases, se particionan los datos y se repite el procedimiento sobre cada subconjunto hasta cumplir una condición de parada [2]. Entre los criterios de partición más utilizados se encuentran la ganancia de información, la razón de ganancia y el índice de Gini. La ganancia de información mide la reducción de entropía después de una división, mientras que el índice de Gini evalúa la impureza de los nodos.

Una de sus principales ventajas es la interpretabilidad, ya que el árbol puede leerse como un conjunto de reglas del tipo “si-entonces”. Esto permite identificar qué variables influyen en la clasificación y bajo qué condiciones se asigna una clase específica. Además, los Árboles de Decisión pueden manejar variables numéricas y categóricas, y generalmente requieren menos preparación que métodos basados en distancia, pues no dependen de la escala de las variables.

No obstante, estos modelos pueden sobreajustarse cuando el árbol crece demasiado y aprende patrones específicos del conjunto de entrenamiento. Para controlar esta limitación se utilizan hiperparámetros como `max_depth`, `min_samples_split`, `min_samples_leaf` o técnicas de poda. También debe considerarse el costo computacional, especialmente en conjuntos con muchas observaciones o atributos, ya que algoritmos clásicos como C4.5 pueden presentar problemas de escalabilidad [2].

Los Árboles de Decisión también constituyen la base de métodos de ensamble como Random Forest, AdaBoost y XGBoost. Freund y Mason [1] muestran cómo las ideas de árboles y boosting pueden combinarse mediante los *Alternating Decision Trees*, manteniendo una lógica basada en reglas ponderadas. En el laboratorio, este algoritmo permitirá construir modelos interpretables para los datos de Potabilidad y Diabetes, evaluando tanto una configuración estándar como variaciones de profundidad y número mínimo de muestras por división.

5. Análisis experimental del algoritmo Árboles de Decisión

En esta sección se presenta el análisis experimental del algoritmo de Árboles de Decisión aplicado al conjunto de datos *BankChurners*. Se empleó la clase `Supervisado` del paquete pythónico del curso, extendida con los métodos correspondientes para este algoritmo.

5.1. Preparación de los datos

A diferencia de KNN, los Árboles de Decisión no requieren estandarización de las variables predictoras, puesto que las divisiones se basan en umbrales sobre cada variable individual y no dependen de la escala ni de distancias entre observaciones [2]. Por este motivo, la preparación de datos para este algoritmo omitió el paso de `StandardScaler`, manteniendo únicamente la codificación numérica de las variables categóricas mediante `LabelEncoder` y la partición estratificada 75/25 con semilla fija.

El conjunto de datos conserva las mismas 14 variables predictoras y la variable objetivo binaria (*Attrition_Flag*), con el desbalance ya descrito de 687 clientes activos (84.8 %) frente a 123 que abandonaron (15.2 %).

5.2. Configuración estándar del Árbol de Decisión

La configuración estándar del clasificador `DecisionTreeClassifier` de scikit-learn se definió con los valores por defecto: criterio de partición Gini, profundidad máxima sin restricción (`max_depth=None`), `min_samples_split=2` y `min_samples_leaf=1`. Esta configuración permite que el árbol crezca libremente hasta que cada hoja sea pura o contenga menos muestras que `min_samples_split`.

Los resultados del modelo estándar se presentan en la Tabla 3.

Cuadro 3: Resultados del modelo DT estándar (Gini, sin restricción de profundidad).

Métrica	Valor
Precisión Global	0.9015
Error Global	0.0985
Precisión – Attrited Customer (clase 0)	0.6774
Precisión – Existing Customer (clase 1)	0.9419
Profundidad del árbol	9
Número de hojas	35

La precisión global de 90.15 % supera la obtenida por el KNN estándar (86.70 %). Además, la precisión de la clase minoritaria (*Attrited Customer*) alcanza 67.74 %, muy por encima del 32.26 % logrado por KNN con pesos uniformes. No obstante, el árbol sin restricción presenta 35 hojas y 9 niveles de profundidad, lo que sugiere un posible sobreajuste que podría afectar la generalización.

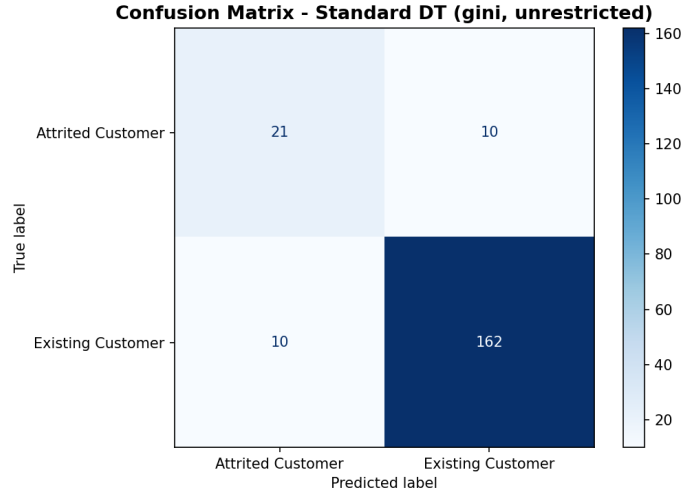


Figura 8: Matriz de confusión del modelo DT estándar (Gini, sin restricción).

5.3. Variaciones de configuración

Se evaluaron 800 configuraciones distintas del algoritmo, combinando los siguientes hiperparámetros:

- **Criterio de partición (criterion):** gini (índice de Gini) y entropy (ganancia de información).
- **Profundidad máxima (max_depth):** de 1 a 20.
- **Mínimo de muestras para dividir (min_samples_split):** 2, 5, 10, 15 y 20.
- **Mínimo de muestras por hoja (min_samples_leaf):** 1, 2, 5 y 10.

La Figura 9 muestra la evolución de la precisión global conforme varía `max_depth`, diferenciada por criterio y valores de `min_samples_split`.

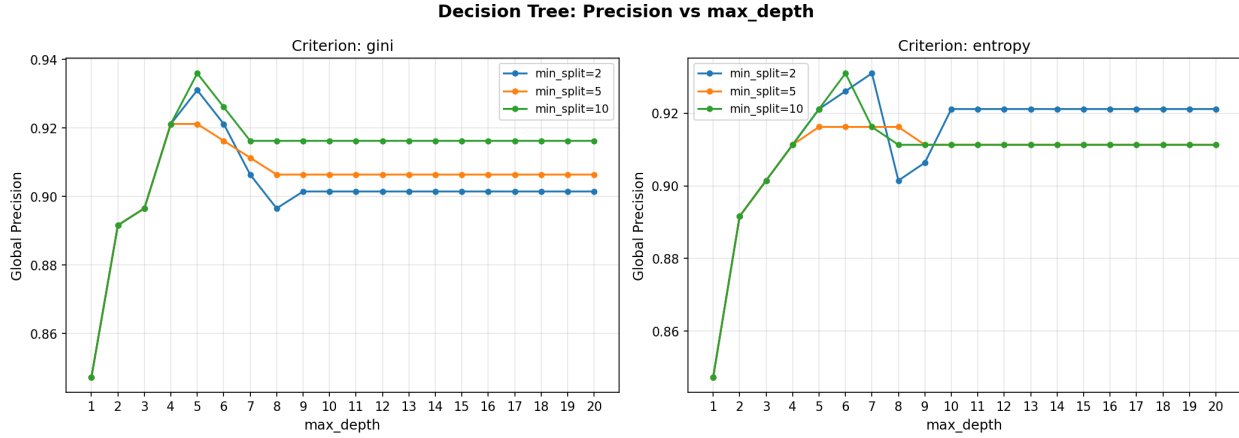


Figura 9: Precisión global en función de la profundidad máxima para distintos valores de `min_samples_split` y cada criterio.

Se observa que la precisión crece rápidamente hasta `max_depth` $\approx 5-7$, donde se estabiliza. Profundidades mayores no aportan mejoras significativas, lo cual concuerda con la teoría de que árboles más profundos tienden al sobreajuste sin mejorar la capacidad predictiva sobre datos nuevos [2].

La Figura 10 presenta un mapa de calor con la precisión para combinaciones de profundidad y `min_samples_split`.

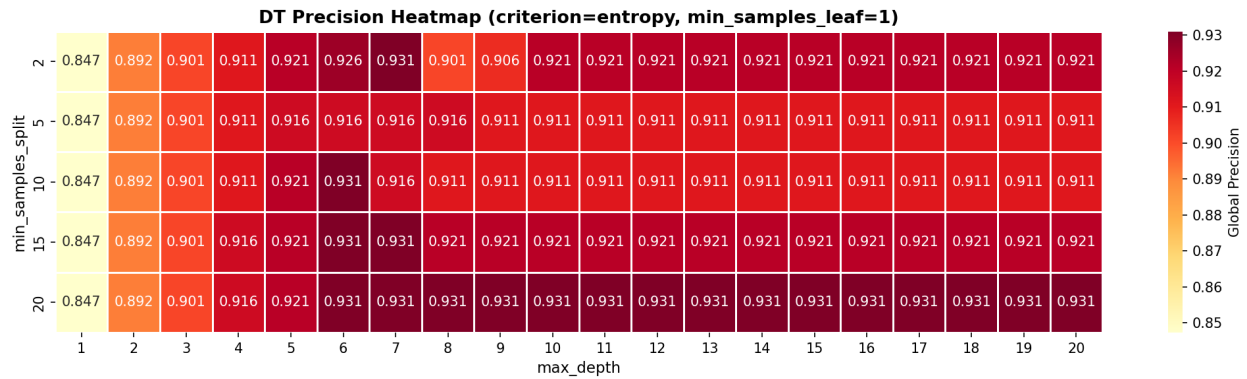


Figura 10: Mapa de calor de precisión global por profundidad y `min_samples_split` (mejor criterio, `min_samples_leaf=1`).

5.4. Efecto de `min_samples_leaf`

El parámetro `min_samples_leaf` controla el número mínimo de muestras que debe contener cada hoja del árbol. Valores mayores actúan como mecanismo de regularización, forzando hojas más generales. La Figura 11 muestra su efecto.

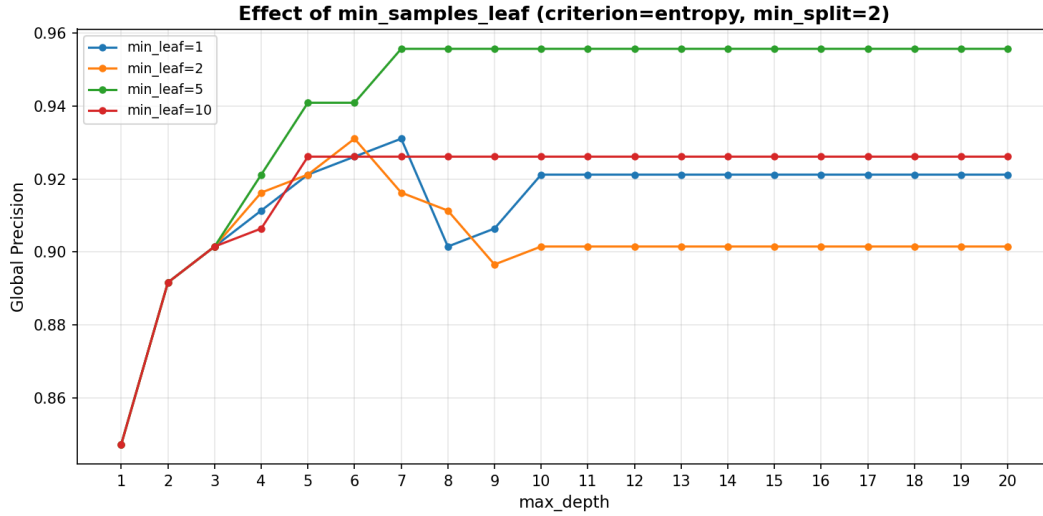


Figura 11: Efecto de `min_samples_leaf` sobre la precisión para distintas profundidades.

Se observa que `min_samples_leaf=5` produce los mejores resultados globales, actuando como una poda suave que evita hojas con muy pocas muestras y reduce el sobreajuste. Con `min_samples_leaf=10` la regularización es excesiva, perdiendo capacidad discriminativa.

5.5. Resumen del rendimiento por profundidad

La Figura 12 sintetiza el rendimiento agrupado por `max_depth`.

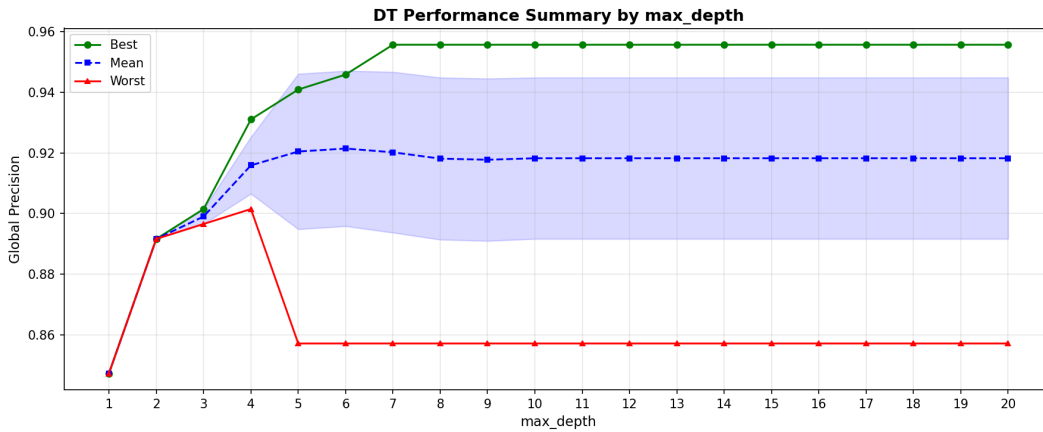


Figura 12: Resumen del rendimiento de DT por profundidad: mejor, media y peor precisión global.

El rango óptimo se ubica entre `max_depth` 5 y 7. A partir de `max_depth=7`, la mejor precisión se estabiliza en 95.57%, indicando que el árbol ya ha capturado las reglas de decisión principales.

5.6. Selección del mejor modelo

La Figura 13 presenta las diez mejores configuraciones.

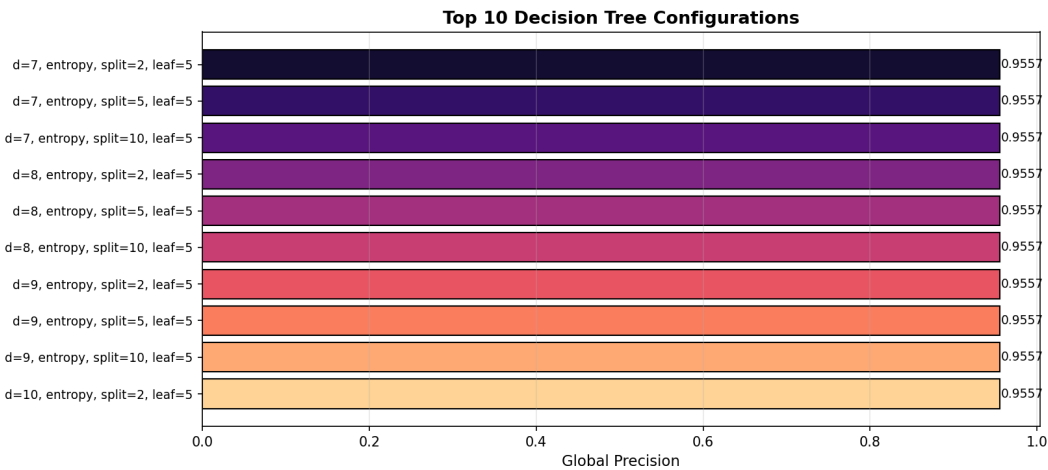


Figura 13: Top 10 configuraciones de Árbol de Decisión por precisión global.

El mejor modelo corresponde a la configuración: criterio `entropy`, `max_depth=7`, `min_samples_split=2` y `min_samples_leaf=5`, con una precisión global de **95.57 %**. Todas las configuraciones del top 10 comparten el criterio `entropy` y `min_samples_leaf=5`, confirmando que esta combinación proporciona la mejor regularización.

5.7. Análisis del mejor modelo

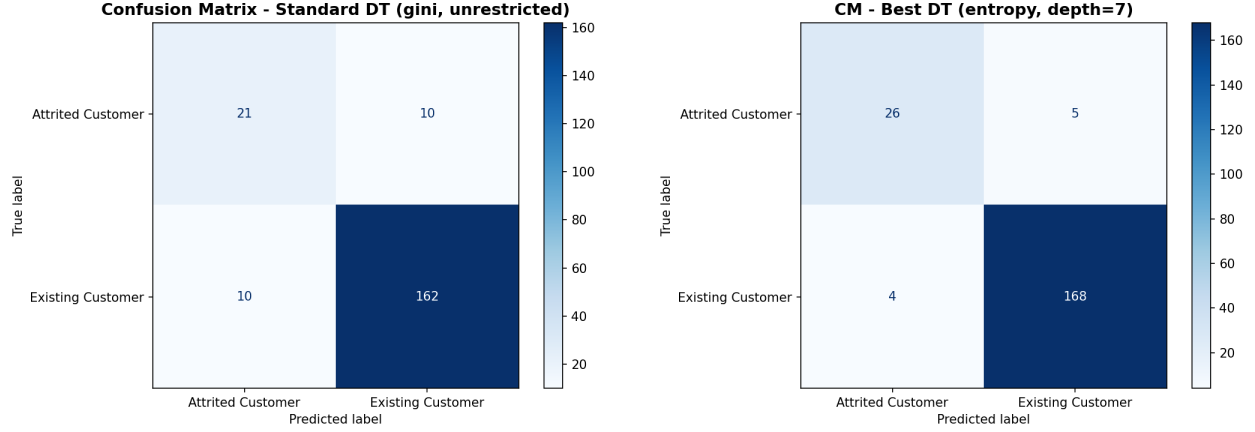
La Tabla 4 compara la configuración estándar con la mejor configuración.

Cuadro 4: Comparación entre la configuración estándar y la mejor configuración de DT.

Métrica	Estándar (gini, sin restricción)	Mejor modelo (entropy, depth=7, leaf=5)
Precisión Global	0.9015	0.9557
Error Global	0.0985	0.0443
Precisión – Attrited (clase 0)	0.6774	0.8387
Precisión – Existing (clase 1)	0.9419	0.9767
Profundidad	9	7
Hojas	35	22

El mejor modelo logra una mejora de 5.42 puntos porcentuales en precisión global, reduciendo el error de 9.85 % a 4.43 %. La precisión de la clase minoritaria mejoró de 67.74 % a

83.87%, y la de la clase mayoritaria de 94.19% a 97.67%. Notablemente, el modelo optimizado tiene menor profundidad (7 vs. 9) y menos hojas (22 vs. 35) que el estándar, lo cual indica una mejor generalización con menor complejidad.



(a) Modelo estándar (gini, sin restricción).

(b) Mejor modelo (entropy, depth=7, leaf=5).

Figura 14: Matrices de confusión: configuración estándar vs. mejor modelo de DT.

El reporte de clasificación del mejor modelo muestra un *F1-score* de 0.85 para la clase minoritaria y 0.97 para la mayoritaria, con una precisión ponderada de 96%.

5.8. Importancia de las variables

Una ventaja fundamental de los Árboles de Decisión es la posibilidad de cuantificar la importancia de cada variable predictora. La Figura 15 presenta la importancia de las variables en el mejor modelo.

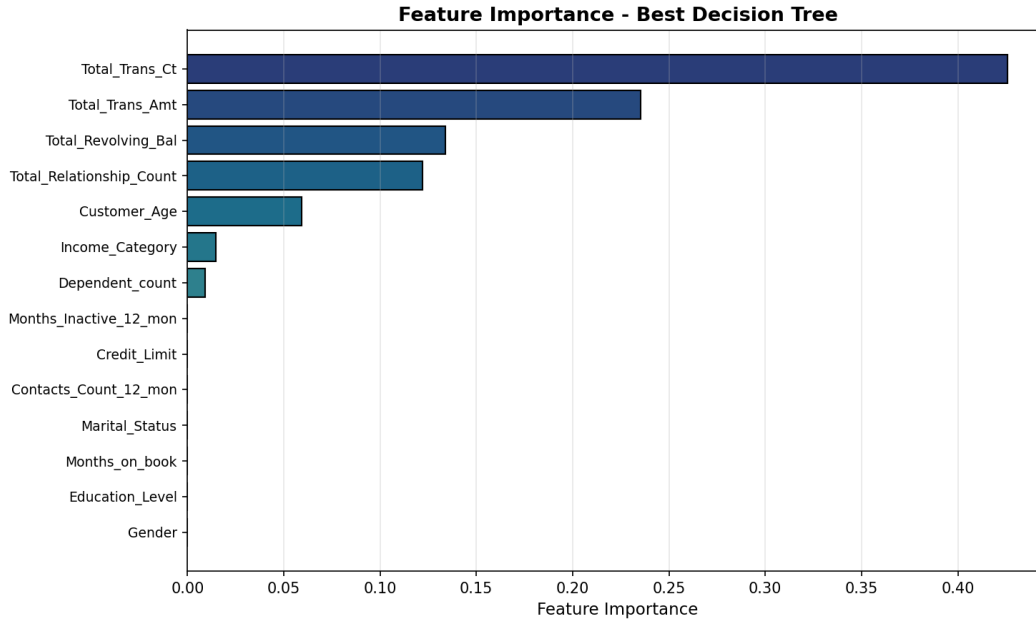


Figura 15: Importancia de las variables en el mejor modelo de Árbol de Decisión.

Las variables más relevantes para predecir la deserción de clientes son **Total_Trans_Ct** (número total de transacciones, 42.5%), **Total_Trans_Amt** (monto total de transacciones, 23.8%) y **Total_Revolving_Bal** (saldo rotativo, 13.8%). Estas tres variables concentran el 80 % de la capacidad discriminativa del modelo, lo cual resulta coherente desde el punto de vista del negocio: los clientes que realizan pocas transacciones y mantienen bajos saldos rotativos tienen mayor probabilidad de abandonar el servicio.

5.9. Estructura del árbol

La Figura 16 muestra los primeros cuatro niveles del árbol óptimo, permitiendo visualizar las reglas de decisión principales.

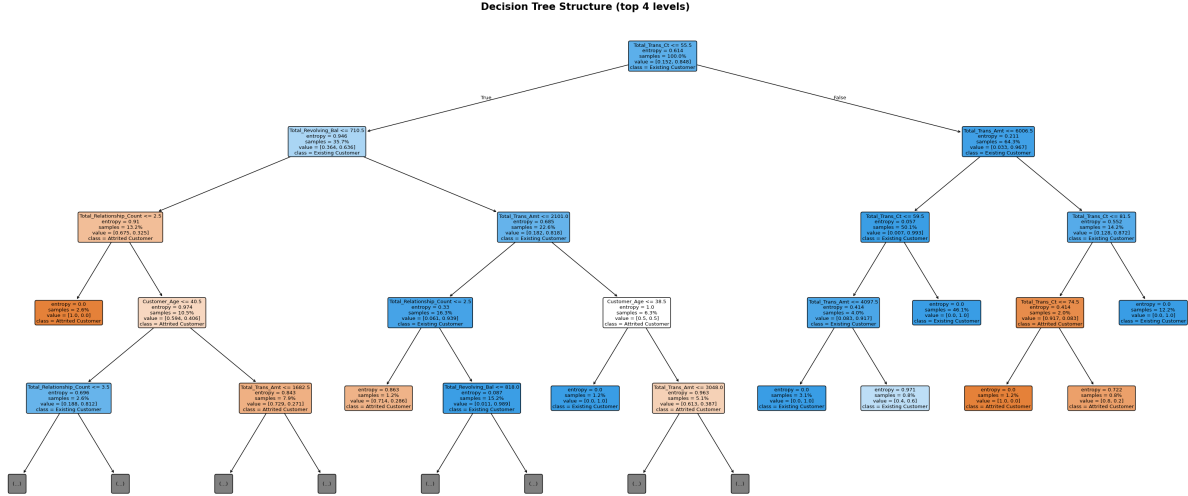


Figura 16: Estructura del mejor Árbol de Decisión (primeros 4 niveles).

5.10. Interpretación de resultados

1. **Criterio de partición:** El criterio **entropy** (ganancia de información) produjo mejores resultados que **gini** en este conjunto de datos. La entropía tiende a generar particiones más equilibradas, lo cual resulta beneficioso en datos con clases desbalanceadas.
2. **Profundidad óptima:** El valor **max_depth=7** representa el equilibrio entre capacidad expresiva y generalización. Árboles más profundos capturan patrones demasiado específicos del entrenamiento sin mejorar el rendimiento en datos nuevos, conforme advierte Su y Zhang [2] sobre la escalabilidad y sobreajuste de los algoritmos clásicos.
3. **Regularización mediante min_samples_leaf:** Este parámetro resultó determinante. Con **min_samples_leaf=5**, cada hoja contiene al menos 5 muestras, evitando decisiones basadas en muy pocas observaciones. Esto funciona como una técnica de poda implícita, alternativa a las técnicas de poda post-entrenamiento.
4. **Interpretabilidad:** A diferencia de KNN, el Árbol de Decisión permite identificar las variables más relevantes y las reglas de decisión aprendidas. Esto facilita la comunicación de resultados y la validación del modelo con expertos del dominio.
5. **Superioridad frente a KNN:** El mejor DT (95.57 %) supera significativamente al mejor KNN (90.64 %), tanto en precisión global como en precisión de la clase minoritaria (83.87 % vs. 41.94 %). Esta ventaja se explica por la capacidad del árbol de generar fronteras de decisión alineadas con los ejes, lo cual resulta más efectivo

cuando pocas variables dominan la clasificación, como ocurre con `Total_Trans_Ct` y `Total_Trans_Amt`.

6. Random Forest (RF)

6.1. Definición

El *Random Forest* (Bosque Aleatorio) es un algoritmo de aprendizaje automático basado en el enfoque de *ensemble learning*, el cual construye múltiples árboles de decisión y combina sus resultados para mejorar la precisión y reducir el sobreajuste. Este método permite modelar relaciones complejas entre variables sin asumir linealidad ni independencia entre ellas [?].

6.2. Funcionamiento general

El algoritmo Random Forest se fundamenta en tres principios:

- **Construcción de múltiples árboles:** Se generan varios árboles de decisión utilizando diferentes muestras de los datos.
- **Selección aleatoria de variables:** En cada nodo del árbol se selecciona un subconjunto aleatorio de variables para realizar la partición.
- **Agregación de resultados:** Las predicciones de todos los árboles se combinan. En regresión se utiliza el promedio, mientras que en clasificación se emplea votación mayoritaria.

La estimación final se obtiene como el promedio de las predicciones generadas por todos los árboles del bosque [?].

6.3. Proceso del algoritmo

El procedimiento general del Random Forest es el siguiente:

1. Se generan múltiples muestras *bootstrap* a partir del conjunto de datos original.
2. Para cada muestra:
 - Se construye un árbol de decisión.
 - En cada nodo:

- Se selecciona aleatoriamente un subconjunto de variables.
 - Se determina la mejor división según una función objetivo.
3. Se repite el proceso para todos los árboles.
 4. Se combinan los resultados para obtener la predicción final.

6.4. Función objetivo

En los enfoques tradicionales, el Random Forest utiliza el error cuadrático medio (MSE) en problemas de regresión. Sin embargo, el documento propone el uso de una función de probabilidad basada en una distribución beta-binomial como criterio de división de los datos. Este enfoque permite modelar mejor estructuras probabilísticas complejas y mejorar la precisión en estimaciones [?].

6.5. Ventajas

Entre las principales ventajas del Random Forest se encuentran:

- Reduce el sobreajuste.
- Maneja relaciones no lineales.
- No requiere supuestos estadísticos fuertes.
- Es robusto ante ruido en los datos.
- Puede trabajar con grandes cantidades de variables.

6.6. Limitaciones

Uno de los principales problemas del Random Forest es su desempeño con datos desbalanceados. En estos casos, el muestreo *bootstrap* puede no representar adecuadamente las observaciones más importantes, generando sesgos en las predicciones [?].

6.7. Mejora mediante Amplified Bootstrap

Para resolver el problema de desbalance, el documento propone el uso del *Amplified Bootstrap*, el cual consiste en:

- Replicar observaciones importantes.

- Ajustar pesos para mantener la validez estadística.
- Mejorar la representación de datos extremos.

Este método permite reducir el sesgo y mejorar la precisión del modelo en conjuntos de datos desbalanceados [?].

6.8. Aplicaciones

El Random Forest se aplica en diversos campos, tales como:

- Clasificación y regresión.
- Análisis de datos complejos.
- Estimación de poblaciones en estudios estadísticos.

En el documento analizado, se utiliza para estimar poblaciones minoritarias lingüísticas, mostrando mejores resultados que modelos estadísticos tradicionales [?].

6.9. Conclusión

El Random Forest es una técnica poderosa que combina múltiples árboles de decisión para obtener predicciones más precisas y robustas. Su capacidad para manejar datos complejos y su mejora mediante técnicas como el *Amplified Bootstrap* lo convierten en una herramienta fundamental en el análisis de datos moderno.

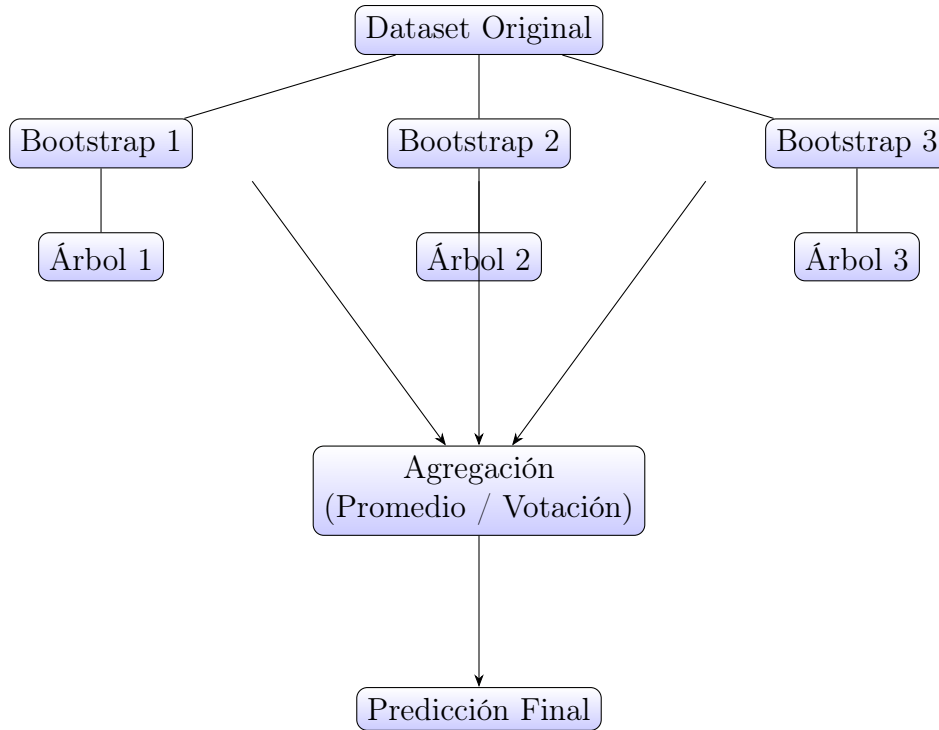


Figura 17: Estructura del algoritmo Random Forest

7. Análisis experimental del algoritmo Random Forest

En esta sección se presenta el análisis experimental del algoritmo Random Forest aplicado al conjunto de datos *BankChurners*. El objetivo es evaluar el desempeño del modelo bajo una configuración estándar y distintas variaciones de hiperparámetros, con el fin de seleccionar el mejor modelo y analizar su comportamiento.

7.1. Preparación de los datos

El conjunto de datos fue preprocesado de forma consistente con los modelos anteriores. Se realizó la codificación de la variable objetivo *Attrition_Flag* en formato binario (0: *Existing Customer*, 1: *Attrited Customer*). Asimismo, las variables categóricas fueron transformadas mediante `get_dummies`, generando representaciones numéricas adecuadas para el modelo.

A diferencia de KNN, el algoritmo Random Forest no requiere estandarización de las variables, ya que los árboles de decisión son invariantes a la escala. Sin embargo, se mantuvo este paso para asegurar consistencia metodológica entre los modelos. Finalmente, los datos se dividieron en un 75 % para entrenamiento y un 25 % para prueba.

7.2. Configuración estándar

La configuración estándar del modelo se definió utilizando el clasificador `RandomForestClassifier` de *scikit-learn*, con los siguientes parámetros:

- `n_estimators = 100`
- `max_depth = None`
- `min_samples_split = 2`

Esta configuración permite construir un conjunto de árboles sin restricción de profundidad, lo cual puede aumentar la capacidad de aprendizaje, pero también el riesgo de sobreajuste.

7.3. Variaciones de configuración

Se evaluaron distintas configuraciones del modelo variando principalmente:

- **Número de árboles (`n_estimators`):** 50, 100, 200 y 300.
- **Profundidad máxima (`max_depth`):** valores de 5, 10, 20 y sin restricción.

Los resultados obtenidos se muestran a continuación:

Cuadro 5: Resultados de distintas configuraciones de Random Forest.

Configuración	Precisión
50 árboles, sin profundidad	0.9310
100 árboles, profundidad 10	0.9212
200 árboles, profundidad 20	0.9310
300 árboles, sin profundidad	0.9310
100 árboles, profundidad 5	0.9015

Se observa que el rendimiento del modelo es relativamente estable para configuraciones con mayor número de árboles y sin restricción de profundidad. En contraste, limitar excesivamente la profundidad reduce la capacidad predictiva.

7.4. Selección del mejor modelo

El mejor modelo corresponde a la siguiente configuración:

- `n_estimators = 50`
- `max_depth = None`

Este modelo alcanzó una precisión global de **93.10 %**, mostrando un equilibrio adecuado entre complejidad y desempeño.

7.5. Análisis del mejor modelo

La matriz de confusión del mejor modelo se presenta en la Figura 18.

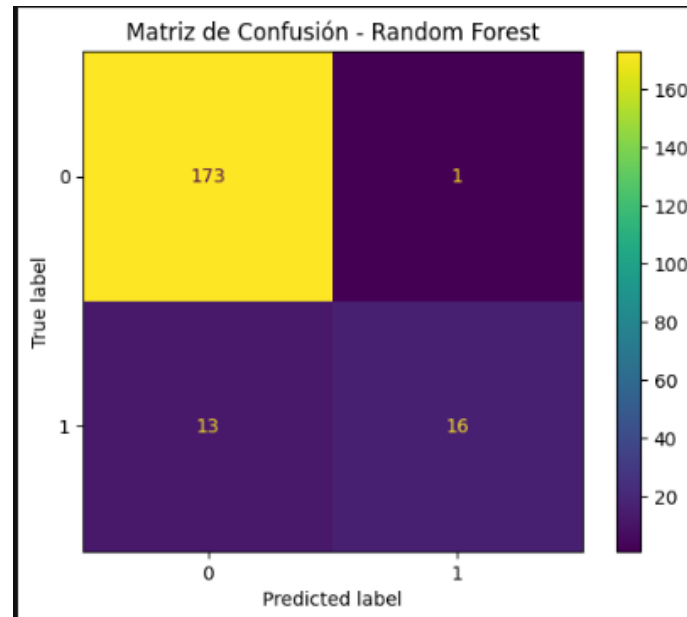


Figura 18: Matriz de confusión del mejor modelo Random Forest.

A partir de la matriz de confusión, se obtiene la siguiente distribución:

- Verdaderos negativos: 173
- Falsos positivos: 1
- Falsos negativos: 13
- Verdaderos positivos: 16

La precisión global del modelo es de 93.10 %, lo cual indica un desempeño sólido en términos generales.

Cuadro 6: Métricas del mejor modelo Random Forest.

Métrica	Valor
Precisión Global	0.9310
Error Global	0.0690
Precisión – Existing Customer (clase 0)	0.93
Precisión – Attrited Customer (clase 1)	0.94

Si bien el modelo presenta una alta precisión en ambas clases, es importante analizar el *recall* de la clase minoritaria. En este caso, el modelo identifica correctamente aproximadamente el 55 % de los clientes que abandonan el servicio, lo cual evidencia una limitación en la detección de esta clase.

7.6. Importancia de variables

El modelo Random Forest permite identificar la relevancia de cada variable en el proceso de clasificación. La Figura 19 muestra la importancia de las variables en el mejor modelo.

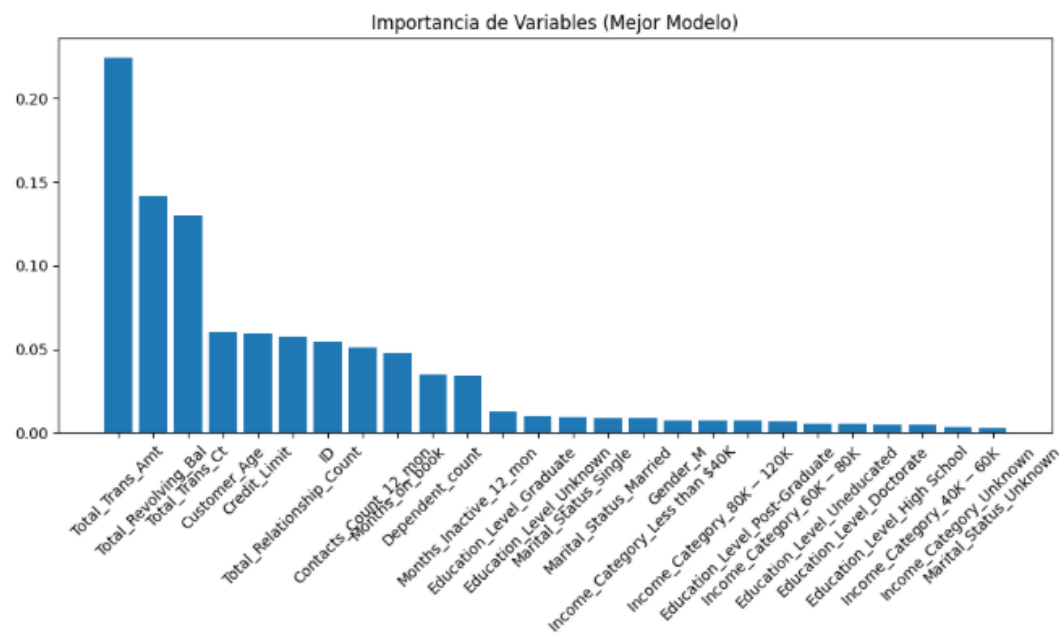


Figura 19: Importancia de variables en el mejor modelo Random Forest.

Las variables más importantes corresponden a:

- Total_Trans_Amt
- Total_Revolving_Bal
- Customer_Age

Esto indica que el comportamiento transaccional del cliente es un factor determinante en la predicción de abandono.

7.7. Interpretación de resultados

A partir del análisis experimental, se destacan los siguientes puntos:

1. **Estabilidad del modelo:** Random Forest presenta un desempeño estable frente a cambios en el número de árboles, siempre que no se limite excesivamente la profundidad.
2. **Control del sobreajuste:** A diferencia de un árbol de decisión individual, el ensamble reduce el riesgo de sobreajuste mediante la agregación de múltiples modelos.
3. **Desbalance de clases:** El modelo mantiene dificultades para detectar la clase minoritaria, lo cual es consistente con la naturaleza del dataset.
4. **Comparación con otros modelos:** Random Forest supera al KNN en precisión global (93.10 % vs. 90.64 %), pero es inferior al Árbol de Decisión optimizado (95.57 %), especialmente en la detección de la clase minoritaria.
5. **Importancia de variables:** El modelo confirma que las variables relacionadas con la actividad financiera del cliente son las más influyentes en la predicción.

8. AdaBoost

AdaBoost (*Adaptive Boosting*) es un método de ensamble secuencial que combina múltiples clasificadores débiles, usualmente árboles de decisión poco profundos, para construir un clasificador fuerte. Su funcionamiento se basa en asignar mayor peso a las observaciones mal clasificadas en iteraciones previas, de forma que el algoritmo concentre progresivamente el aprendizaje en los casos más difíciles.

Conviene distinguir AdaBoost del concepto de *bootstrap*. Aunque en algunos materiales introductorios se confunden ambos términos, AdaBoost pertenece a la familia de *boosting*, no a la de *bagging*. Su lógica central no es el remuestreo independiente, sino la reponderación adaptativa de observaciones y la combinación secuencial de clasificadores.

Desde la perspectiva analítica, AdaBoost es atractivo porque mejora el desempeño de clasificadores simples, conserva una estructura relativamente interpretable cuando usa árboles base pequeños y permite estudiar claramente el efecto de hiperparámetros como `n_estimators`, `learning_rate` y la profundidad del árbol base.

8.1. XGBoost

XGBoost (*Extreme Gradient Boosting*) es una implementación optimizada del paradigma de *gradient boosting*. A diferencia de AdaBoost, que ajusta pesos sobre observaciones, XGBoost agrega árboles de manera aditiva para corregir gradientes o residuos del modelo anterior respecto a una función de pérdida.

Su relevancia práctica se debe a varias características: regularización explícita, control fino de complejidad, muestreo de filas y columnas, eficiencia computacional y buen desempeño sobre datos tabulares. Estos elementos lo convierten en un algoritmo especialmente adecuado para problemas de clasificación estructurados, como la predicción de churn bancario.

8.2. Mapa conceptual

La Tabla 7 resume los elementos conceptuales más importantes de AdaBoost y XGBoost.

Cuadro 7: Mapa conceptual resumido de AdaBoost y XGBoost.

Aspecto	AdaBoost	XGBoost
Fundamento	Repondera observaciones mal clasificadas y combina clasificadores débiles de forma secuencial.	Ajusta árboles aditivos para corregir gradientes o residuos respecto a una función de pérdida.
Tipo de ensamble	Boosting adaptativo.	Gradient boosting optimizado.
Modelo base usual	Árboles poco profundos.	Árboles de decisión con control de complejidad.
Parámetros clave	<code>n_estimators</code> , <code>learning_rate</code> , profundidad del árbol base.	<code>max_depth</code> , <code>learning_rate</code> , <code>subsample</code> , <code>colsample_bytree</code> , <code>reg_lambda</code> .
Ventaja principal	Simplicidad, buena interpretabilidad y alto desempeño con configuraciones compactas.	Flexibilidad, regularización explícita y gran rendimiento en datos tabulares.
Aplicación en este laboratorio	Comparación de versión estándar, robusta, profunda y con <code>SelectKBest</code> .	Comparación de versión estándar, robusta, regularizada y con <code>SelectKBest</code> .

9. Metodología experimental

9.1. Dataset utilizado

El estudio se desarrolló únicamente con el conjunto de datos `BankChurners.csv`. Después de revisar la estructura original del archivo, se definió `Attrition_Flag` como variable objetivo del problema de clasificación. Esta decisión permitió plantear una tarea binaria orientada a distinguir entre *Existing Customer* y *Attrited Customer*.

Las características generales del dataset fueron las siguientes:

- Número de observaciones: 810.
- Número total de variables originales: 16.
- Variable objetivo: `Attrition_Flag`.
- Variable excluida del modelado: `ID`.

- Variables predictoras finales antes de codificación: 14.
- Valores faltantes detectados: 0.

La distribución de la variable objetivo fue desbalanceada: 687 registros correspondieron a *Existing Customer* y 123 a *Attrited Customer*. Por ello, se priorizaron métricas como F1 y ROC AUC sobre una lectura aislada de la exactitud global.

9.2. Justificación de la exclusión de ID

La columna ID se excluyó antes del entrenamiento porque su función es únicamente identificar de forma única a cada cliente. Este tipo de variable no representa un atributo explicativo del fenómeno de abandono, por lo que su inclusión podría inducir patrones espurios, aumentar la dimensionalidad sin aportar información útil y perjudicar la generalización del modelo. En consecuencia, la exclusión de ID constituyó una decisión metodológica necesaria para evitar sesgos y conservar solo variables con valor predictivo real.

9.3. Preprocesamiento

El flujo experimental contempló las siguientes decisiones metodológicas:

1. Carga del archivo `BankChurners.csv`.
2. Exclusión de la columna ID por ser un identificador.
3. Separación entre variables numéricas y categóricas.
4. Imputación por mediana en variables numéricas y por moda en variables categóricas, aunque el dataset no reportó faltantes.
5. Codificación *one-hot* para variables categóricas.
6. Aplicación opcional de escalamiento (`StandardScaler` o `RobustScaler`) según la configuración.
7. Aplicación opcional de selección de atributos mediante `SelectKBest`.

9.4. Configuración estándar

Se definió una configuración base para cada algoritmo:

- **AdaBoost estándar:** imputación por mediana, sin escalamiento, uso de todas las variables disponibles tras la codificación, árbol base tipo *stump* con `max_depth = 1`, `n_estimators = 100` y `learning_rate = 1.0`.
- **XGBoost estándar:** imputación por mediana, sin escalamiento, uso de todas las variables, `n_estimators = 200`, `learning_rate = 0.10`, `max_depth = 4`, `subsample = 1.0`, `colsample_bytree = 1.0` y `reg_lambda = 1.0`.

Estas configuraciones se tomaron como línea base para comparar el efecto de cambios de preprocesamiento y complejidad del modelo.

9.5. Variaciones analizadas

Además de la configuración estándar, se probaron las siguientes variaciones:

1. **AdaBoost_Robust:** sustitución del escalamiento nulo por `RobustScaler`.
2. **AdaBoost_Profundo:** aumento de la profundidad del árbol base para capturar relaciones más complejas.
3. **AdaBoost_SelectKBest:** selección de las 6 mejores variables mediante `SelectKBest` con escalamiento estándar.
4. **XGBoost_Robust:** aplicación de `RobustScaler` manteniendo el resto de la configuración.
5. **XGBoost_Regularizado:** incremento del control de complejidad mediante mayor regularización.
6. **XGBoost_SelectKBest:** selección de las 6 mejores variables con escalamiento estándar.

9.6. Estrategia de evaluación

La comparación de modelos se realizó con validación cruzada estratificada de cinco particiones sobre el conjunto de entrenamiento. Posteriormente, el mejor modelo se reajustó sobre el entrenamiento completo y se evaluó en el conjunto de prueba. El criterio principal de selección siguió el siguiente orden:

1. mayor `cv_test_f1`,
2. desempate por `cv_test_roc_auc`,
3. desempate final por `test_f1`.

10. Resultados del experimento

10.1. Benchmark general

La Tabla 8 resume el desempeño de las ocho configuraciones evaluadas. Los resultados muestran que el comportamiento general fue alto para ambos algoritmos, pero las mejores puntuaciones de validación cruzada se concentraron en `AdaBoost_Profundo`, mientras que el mejor resultado en prueba final correspondió a `AdaBoost_Standard`.

Cuadro 8: Benchmark comparativo de modelos en `BankChurners.csv`.

Modelo	CV F1	CV ROC AUC	Test F1	Test ROC AUC
AdaBoost_Profundo	0.9673	0.9570	0.9709	0.9876
XGBoost_Robust	0.9635	0.9519	0.9685	0.9865
XGBoost_Standard	0.9634	0.9466	0.9683	0.9844
AdaBoost_Standard	0.9615	0.9440	0.9713	0.9807
XGBoost_Regularizado	0.9568	0.9461	0.9602	0.9811
XGBoost_SelectKBest	0.9558	0.9413	0.9507	0.9602
AdaBoost_Robust	0.9554	0.9435	0.9602	0.9837
AdaBoost_SelectKBest	0.9516	0.9396	0.9548	0.9678

10.2. Modelo con mejor validación cruzada y mejor prueba

Para responder de forma explícita a las preguntas clave del informe final, la Tabla 9 resume los modelos ganadores según las métricas principales.

Cuadro 9: Modelos con mejor desempeño según la métrica analizada.

Métrica	Modelo ganador	Valor
Mejor CV F1	AdaBoost_Profundo	0.9673
Mejor CV ROC AUC	AdaBoost_Profundo	0.9570
Mejor Test F1	AdaBoost_Standard	0.9713

Aunque `AdaBoost_Standard` obtuvo un *Test F1* ligeramente superior, el modelo seleccionado como mejor configuración global fue `AdaBoost_Profundo`, ya que lideró simultáneamente el *CV F1* y el *CV ROC AUC*. Esto es consistente con un criterio de selección centrado en la robustez de la validación cruzada y no únicamente en el desempeño de una sola partición de prueba.

10.3. Mejor configuración por algoritmo

La Tabla 10 muestra el mejor representante de cada familia de *boosting*.

Cuadro 10: Mejor configuración por algoritmo en `BankChurners.csv`.

Modelo	CV F1	CV ROC AUC	Test F1
AdaBoost_Profundo	0.9673	0.9570	0.9709
XGBoost_Robust	0.9635	0.9519	0.9685

Los resultados sugieren que AdaBoost fue ligeramente superior a XGBoost en este conjunto de datos específico. Además, la variación profunda de AdaBoost superó tanto a su versión estándar como a las variantes con escalamiento robusto y selección de atributos.

11. Análisis del mejor modelo

11.1. Resumen de desempeño de AdaBoost_Profundo

El mejor modelo global fue `AdaBoost_Profundo`. En validación cruzada obtuvo *accuracy* de 0.9440, *balanced accuracy* de 0.8696, *precision* de 0.9583, *recall* de 0.9767, *F1* de 0.9673 y *ROC AUC* de 0.9570. En el conjunto de prueba alcanzó *accuracy* de 0.9507, *balanced accuracy* de 0.9048, *precision* de 0.9709, *recall* de 0.9709, *F1* de 0.9709 y *ROC AUC* de 0.9876.

Estos resultados indican un modelo muy estable, con diferencias pequeñas entre entrenamiento validado y prueba final. La combinación de un F1 alto y un ROC AUC cercano a 1 evidencia una buena capacidad discriminativa, incluso en presencia de desbalance de clases.

11.2. Matriz de confusión e interpretación

La matriz de confusión del mejor modelo se resume en la Tabla 11. De los 31 clientes que realmente abandonaron, el modelo identificó correctamente 26 y confundió 5 con clientes activos. De los 172 clientes existentes, clasificó correctamente 167 y solo cometió 5 falsos positivos.

Cuadro 11: Matriz de confusión de `AdaBoost_Profundo`.

Clase real	Pred. Attrited	Pred. Existing
Attrited Customer	26	5
Existing Customer	5	167

En términos prácticos, esto significa que el modelo conserva un equilibrio razonable entre sensibilidad al abandono y control de falsas alarmas. No obstante, el desempeño sobre la clase minoritaria sigue siendo más exigente: para *Attrited Customer* el *precision*, *recall* y *F1-score* fueron 0.8387, mientras que para *Existing Customer* alcanzaron 0.9709. Este contraste confirma el efecto del desbalance de clases.

11.3. Variables más influyentes

El gráfico de importancia de variables mostró que las características más influyentes del mejor modelo fueron:

1. `num__Total_Trans_Amt` (0.3614),
2. `num__Total_Trans_Ct` (0.2319),
3. `num__Customer_Age` (0.0897),
4. `num__Total_Revolving_Bal` (0.0683),
5. `num__Total_Relationship_Count` (0.0527).

En un segundo nivel de influencia también destacaron `Credit_Limit`, `Contacts_Count_12_mon` y `Months_on_book`. En conjunto, estos resultados sugieren que el abandono del cliente estuvo fuertemente asociado con el nivel de actividad transaccional, el uso del crédito y la intensidad de la relación del cliente con la entidad bancaria. Desde una perspectiva de negocio, esto es coherente: un cliente que reduce su monto total transado o su número de transacciones puede estar mostrando señales tempranas de desvinculación.

12. Apoyo gráfico para el análisis

A continuación se integran las salidas gráficas principales del experimento.

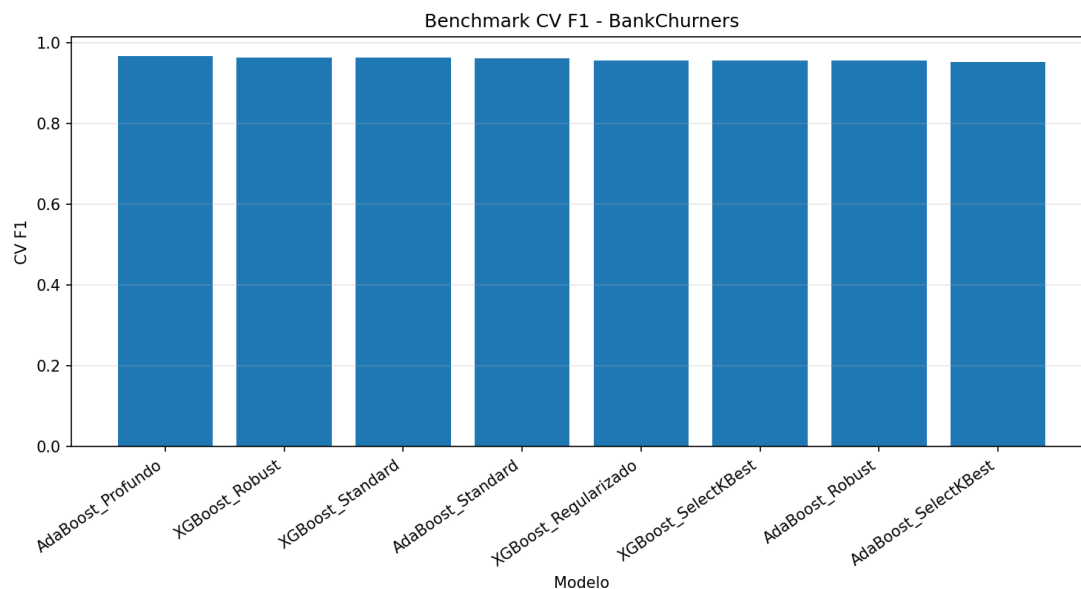


Figura 20: Comparación de modelos según CV F1 en `BankChurners.csv`.

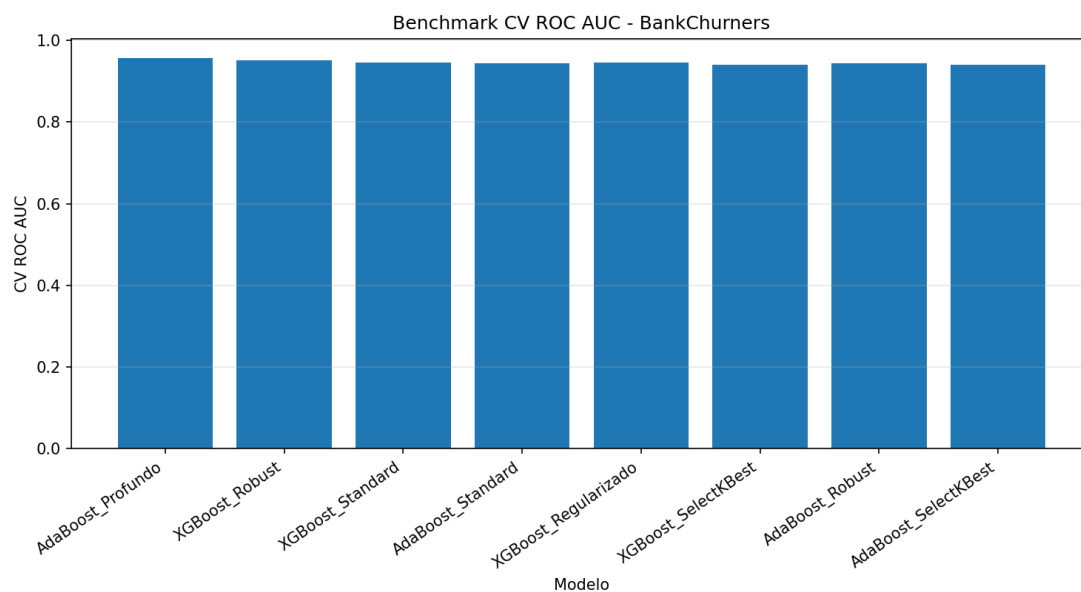


Figura 21: Comparación de modelos según CV ROC AUC en `BankChurners.csv`.

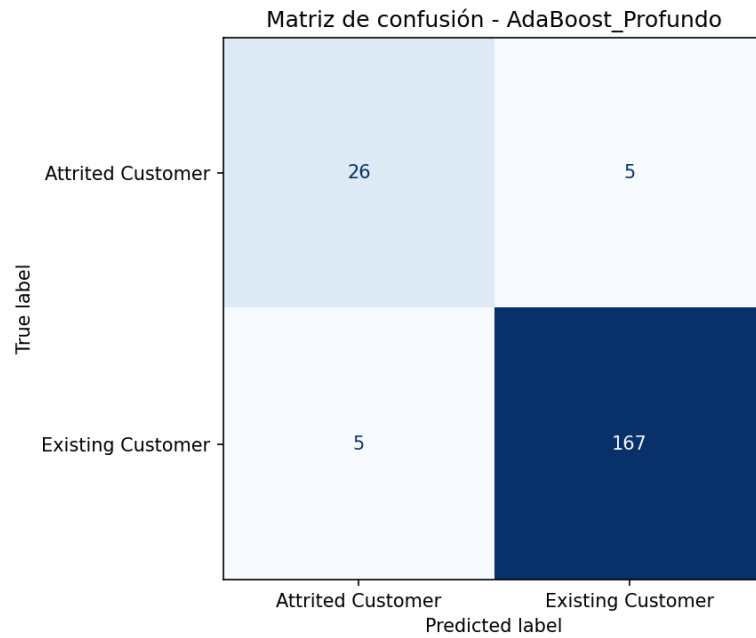


Figura 22: Matriz de confusión del modelo AdaBoost_Profundo.

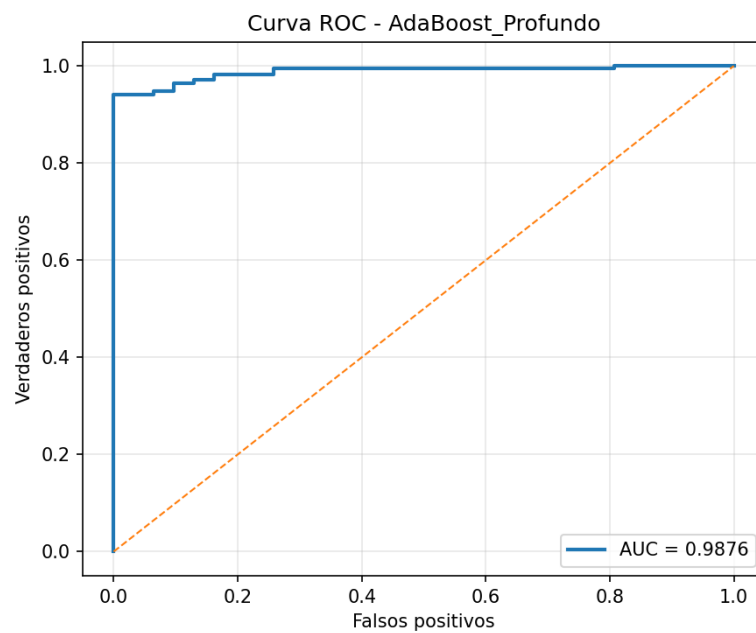


Figura 23: Curva ROC del modelo AdaBoost_Profundo con AUC de 0.9876 en prueba.

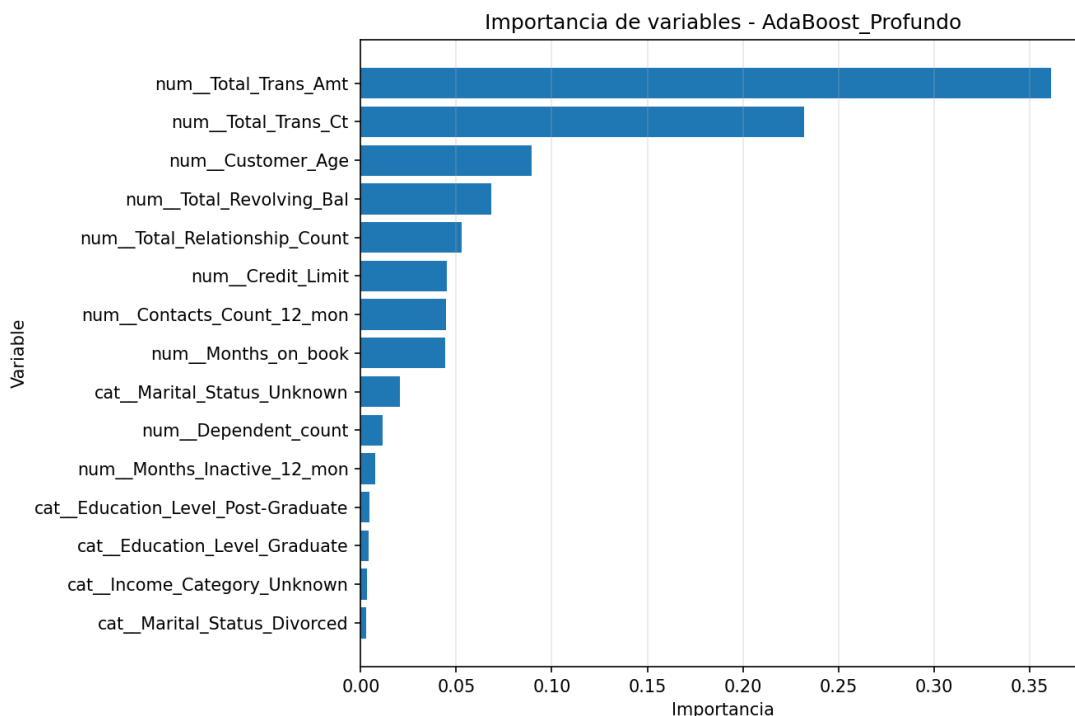


Figura 24: Importancia de variables del modelo `AdaBoost_Profundo`.

13. Discusión

Los resultados permiten responder de manera directa a los elementos que conviene destacar en el documento final. En primer lugar, el dataset utilizado fue `BankChurners.csv` y la variable objetivo fue `Attrition_Flag`. En segundo lugar, la columna `ID` se eliminó por tratarse de un identificador sin valor explicativo, cuya presencia podría introducir ruido en el proceso de aprendizaje.

En tercer lugar, la configuración estándar quedó definida por el uso de imputación por mediana, codificación *one-hot*, todas las variables disponibles tras el preprocesamiento y, según el algoritmo, sus hiperparámetros base. A partir de esa línea base se estudiaron variaciones de escalamiento robusto, incremento de profundidad, regularización y selección de atributos. Esta comparación fue útil para identificar qué modificaciones realmente aportaron mejoras y cuáles redujeron el desempeño.

En cuarto lugar, el mejor comportamiento en validación cruzada fue alcanzado por `AdaBoost_Profundo`, tanto en *CV F1* como en *CV ROC AUC*. Sin embargo, el mayor *Test F1* correspondió a `AdaBoost_Standard`. Esta diferencia sugiere que ambas configuraciones fueron muy competitivas y que el modelo profundo ofreció la mejor combinación de estabilidad y capacidad discriminativa, mientras que el estándar mostró una ligera ventaja puntual

en el conjunto de prueba.

Finalmente, el análisis de importancia de variables indicó que el abandono estuvo muy vinculado a variables transaccionales, especialmente `Total_Trans_Amt` y `Total_Trans_Ct`. Esto tiene sentido desde la lógica del problema: una disminución en la actividad bancaria puede anticipar la salida del cliente. También resultó razonable la presencia de variables como `Customer_Age`, `Total_Revolving_Bal`, `Total_Relationship_Count` y `Credit_Limit`, ya que describen tanto la intensidad del vínculo como el uso financiero del cliente.

14. Conclusiones

El laboratorio permitió comparar dos estrategias de *boosting* sobre un problema realista de clasificación binaria orientado al churn bancario. El experimento confirmó que `BankChurners.csv` puede modelarse de forma efectiva utilizando `Attrition_Flag` como variable objetivo y excluyendo ID del proceso de entrenamiento.

En términos de resultados, `AdaBoost_Profundo` fue la mejor configuración global, ya que obtuvo el mejor *CV F1* (0.9673) y el mejor *CV ROC AUC* (0.9570), además de un *Test F1* muy alto (0.9709). Por su parte, `AdaBoost_Standard` logró el mejor *Test F1* del benchmark con 0.9713, lo que evidencia que la línea base también fue muy competitiva. Entre los modelos XGBoost, la variante `XGBoost_Robust` fue la mejor, aunque quedó ligeramente por debajo del mejor AdaBoost.

Desde la interpretación del modelo, las variables más influyentes fueron `Total_Trans_Amt`, `Total_Trans_Ct`, `Customer_Age`, `Total_Revolving_Bal` y `Total_Relationship_Count`. Esto respalda la idea de que la actividad transaccional y la relación financiera del cliente son señales centrales para anticipar la deserción.

En síntesis, el estudio no solo permitió comparar AdaBoost y XGBoost, sino también documentar con evidencia qué configuraciones funcionaron mejor, por qué se descartó la columna identificadora y cuáles atributos explicaron en mayor medida la predicción final del abandono de clientes.

Referencias

- [1] Freund, Y., & Mason, L. (1999). The alternating decision tree learning algorithm. *Proceedings of the Sixteenth International Conference on Machine Learning*, 124–133.
- [2] Su, J., & Zhang, H. (2006). A fast decision tree learning algorithm. *Proceedings of the National Conference on Artificial Intelligence*, 500–505.

- [3] Zhang, S. (2022). Challenges in KNN classification. *IEEE Transactions on Knowledge and Data Engineering*, 34(10), 4663–4675.
- [4] Zhang, S., & Li, J. (2023). KNN classification with one-step computation. *IEEE Transactions on Knowledge and Data Engineering*, 35(3), 2711–2723.
- [5] Zhang, S., Li, X., Zong, M., Zhu, X., & Cheng, D. (2017). Learning k for kNN classification. *ACM Transactions on Intelligent Systems and Technology*, 8(3), 1–19.
- [6] Zhang, S., Li, X., Zong, M., Zhu, X., & Wang, R. (2018). Efficient kNN classification with different numbers of nearest neighbors. *IEEE Transactions on Neural Networks and Learning Systems*, 29(5), 1774–1785.
- [7] Kang, J., Hall, A. C., & Lee, G. (2026). *A random forest approach with amplified bootstrap for counting language minority groups in the United States*. Mathematics, 14(5), 768. <https://doi.org/10.3390/math14050768>
- [8] Breiman, L. (2001). Random forests. *Machine Learning*, 45(1), 5–32. <https://doi.org/10.1023/A:1010933404324>
- [9] Hastie, T., Tibshirani, R., & Friedman, J. (2009). *The elements of statistical learning* (2nd ed.). Springer.